

Codex Verification Agent for RTL Testbench Generation

Mini Project 2 - Phase 3 Report | AI for Design Verification | Topic 2

Arizona State University

1. Motivation for the Problem

Hardware verification is one of the most expensive and time-consuming phases of modern chip design. Industry estimates suggest that verification consumes more than 70 percent of total design effort, and this fraction grows as designs become more complex. Writing self-checking testbenches requires engineers to deeply understand both the hardware specification and the subtleties of a target simulation environment - a combination of skills that is scarce and slow to scale.

Traditional automation tools - such as constrained-random simulation, coverage-driven verification, and formal property checking - each address part of the problem but all require significant manual setup: writing coverage models, defining properties, or encoding assertions by hand. None of these tools can read an English-language specification and produce a working self-checking testbench without expert human involvement.

AI-based agents offer a qualitatively different capability. A large language model can read a natural-language hardware description, reason about intended behavior, and produce Verilog test code. However, naive one-shot prompting is insufficient: the model may generate syntactically valid but semantically wrong assertions, hallucinate timing assumptions the spec does not guarantee, or produce code that is too permissive or too strict. An agent-based approach - where the model operates in a structured loop with feedback from actual EDA simulation - is the correct architecture for this task. This project implements and evaluates exactly that approach.

2. Problem Definition

Task: Given a natural-language hardware specification and a set of 31 candidate RTL implementations, generate a self-checking Verilog testbench that passes exactly the correct implementation and fails all buggy ones. This is the ICLAD Google Verification Benchmark task.

Inputs:

- `specification.md`: English prose description of the hardware module's behavior, interface, and timing.
- `mutant_*.v`: 31 Verilog implementations of the module. Exactly one is fully correct; the others contain injected bugs.

Outputs:

- A self-checking Verilog testbench `tb/<name>_tb.v` that compiles under iverilog -g2012 and runs under vvp.
- Intermediate artifacts: `parsed_spec.json`, `test_plan.json`, simulation logs, and `final_result.json`.

Scoring metric: $\text{Score} = 1 / (\text{number of passing implementations})$. If exactly one implementation passes (the correct one), $\text{Score} = 1.0$. If the testbench is too loose and passes all 31, $\text{Score} = 1/31 = 0.032$. If it is too strict and rejects even the correct design, $\text{Score} = 0$. The objective is to maximize score across all benchmark problems.

3. System Overview

The system is a structured five-stage agent pipeline. The overall data flow is:

```
specification.md → Spec Parser (LLM) → parsed_spec.json → Test Planner (LLM) →  
test_plan.json → Testbench Generator (LLM) → tb/<name>_tb.v → iverilog + vvp → Score →  
Repair / Refinement Loop
```

At each stage the agent produces an intermediate artifact (JSON or Verilog). Simulator output - compile errors, pass/fail per mutant - is fed back into the repair and refinement prompts. The pipeline is fully automated: a single command `python3 run_pipeline.py --problem-dir <path>` executes all stages without any manual steps. The agent's modular design means each stage can be inspected, logged, and reasoned about independently.

4. Agent Architecture

The agent is implemented across four Python modules coordinated by `agent.py`. It differs from one-shot prompting in that each stage reads structured output from the previous stage, the simulator's raw output is parsed and used as feedback, and failure triggers automatic repair rather than requiring human intervention.

Planner / Decision Module (`spec_parser.py`, `tb_generator.py`)

The planner uses Codex to parse `specification.md` into a structured JSON object containing the module interface, behavioral description, and critically, three oracle-planning fields: `guaranteed_properties` (behaviors the spec explicitly promises), `ambiguous_properties` (plausible but unstated behaviors), and `forbidden_assumptions` (things the testbench must never assert). This structured representation is the foundation for all downstream generation. The planner also selects an oracle style: **formula** for combinational modules, **cycle_accurate** for simple sequential modules, and **protocol_invariant** for FIFO/CDC/credit/handshake modules. This routing decision determines which generation prompt is used.

Executor - Tool Invocation (`simulator.py`)

The executor compiles each generated testbench with `iverilog -g2012` and runs it with `vvp` across all 31 mutants. It captures `stdout/stderr` per mutant, parses `PASS/FAIL` signals, and computes the final score. Compile-error output is preserved verbatim for the repair prompt.

Parser - Metrics Extraction (`agent.py`)

After simulation the agent parses the results JSON to determine: how many mutants passed, which ones passed (by name), whether any compile errors occurred, and whether the score meets the target. This parsed state drives the repair/refinement decision.

Feedback Loop / Iteration

Three feedback paths are implemented: (1) If the generated testbench has a compile error, the error message plus the testbench source are sent to `repair_testbench.txt` for a targeted fix. (2) If everything fails at runtime (score = 0), a rescue refinement prompt asks Codex to simplify the oracle rather than rebuild it from scratch. (3) If the guardrail check detects unsafe patterns in a protocol testbench (bounded wait loops, exact counter reconstruction), the agent discards the testbench and requests regeneration before running any simulation. This is fundamentally different from one-shot prompting: the system reads actual tool output, understands why something failed, and uses that understanding to improve.

5. Tools and Frameworks

Tool / Framework	Role in System
gpt-5.4 via Codex CLI	LLM backend for all generation, parsing, repair, and planning stages. Accessed via codex exec in WSL.
Icarus Verilog (iverilog)	EDA simulator. Compiles generated testbenches under SystemVerilog 2012 standard.
VVP runtime (vvp)	Executes compiled Verilog simulations and captures PASS/FAIL output per mutant.
Python 3.10+ (agent.py, etc.)	Custom agent orchestration infrastructure. No third-party agent framework used.
ICLAD Benchmark	10 visible problems used for evaluation; each has specification.md + 31 mutant RTL files.

6. Context Engineering

Context engineering - deciding what information to give the LLM, in what format, and at what stage - is the primary design variable in this project. Raw English specifications are not passed directly to the testbench generator. Instead, Stage 1 converts the spec into a structured JSON object that isolates only what the generator needs: the module interface, guaranteed behaviors, ambiguous behaviors, and forbidden assumptions. This structured intermediary dramatically improves downstream generation quality compared to passing the full raw spec.

Context provided at each stage:

- Stage 1 (parse_spec.txt): full raw specification.md text.
- Stage 2 (gen_testplan.txt): parsed_spec.json - structured, not raw prose.
- Stage 3 (gen_testbench.txt / gen_testbench_protocol.txt): spec text + parsed_spec.json + test_plan.json combined.
- Stage 4 (repair_testbench.txt): generated testbench source + compile error message only.

Context size management:

The JSON extraction in Stage 1 compresses a lengthy spec into a compact structured form, staying well within token limits for Stages 2 and 3. For the one remaining problematic module (cdc_fifo_flops_push_credit), the raw spec itself exceeds practical context bounds, causing the LLM to timeout - this is identified as the primary remaining limitation.

Noise filtering:

The ambiguous_properties and forbidden_assumptions fields in the parsed spec act as an explicit noise filter. They tell the generator which parts of the spec to treat as background context versus which to assert strictly - preventing the most common failure mode (over-confident oracle assertions on unstated behaviors).

7. Improvements from Phase 2

Phase 2 established the basic pipeline: parse spec, generate testbench, simulate, score. Phase 3 introduced five targeted improvements that significantly raised accuracy.

Ambiguity-aware spec parsing:

The parser prompt was redesigned to force Codex to explicitly separate guaranteed, ambiguous, and forbidden behaviors. In Phase 2 the parser produced a flat behavioral summary. The Phase 3 structured output gives downstream stages a clearer signal about what to assert and what to avoid.

Oracle-style routing:

A routing decision now selects between three generation modes (formula, cycle_accurate, protocol_invariant) based on the module type identified in parsing. Phase 2 used a single generic testbench prompt for all modules.

Safe/unsafe check separation:

The test-plan stage now explicitly produces two lists: checks that are safe to assert and checks that are tempting but should be avoided. This prevents the generator from asserting plausible-but-wrong behaviors.

Protocol-specific prompt and guardrail check:

A dedicated prompt (gen_testbench_protocol.txt) handles FIFO/CDC/credit modules with explicit instructions to avoid exact latency and counter assumptions. A structural guardrail check rejects unsafe testbench patterns before simulation and triggers regeneration.

Repair-mode prompt separation:

The repair prompt is now separate from the generation prompt. This prevents repair from blindly regenerating more complexity when the goal is to simplify an over-constrained oracle.

Measured impact:

The counter module, which failed in earlier Phase 2 runs, now consistently produces Score = 1.0 after ambiguity-aware parsing was added. The credit_receiver and fifo_flops protocol modules, which were failing in the Phase 2 batch run, now also score 1.0 following protocol-specific prompt improvements. Overall success rate improved from 7/10 to 9/10.

8. Results

The system was evaluated on all 10 visible ICLAD benchmark problems. Results are shown below. Each problem was run with --max-iters 1 (one generation pass, one repair pass if needed). All 31 mutants were simulated per problem.

Problem	Type	Pass/31	Score	Correct Mutant	Oracle Mode
ecc_sed_encoder	Combinational	1/31	1.0	mutant_18	formula
enc_bin2gray	Combinational	1/31	1.0	mutant_25	formula
enc_bin2onehot	Combinational	1/31	1.0	mutant_27	formula
lfsr	Sequential	1/31	1.0	mutant_29	cycle_accurate
shift_left	Combinational	1/31	1.0	mutant_1	formula

Problem	Type	Pass/31	Score	Correct Mutant	Oracle Mode
shift_right	Combinational	1/31	1.0	mutant_9	formula
counter	Sequential	1/31	1.0	mutant_11	cycle_accurate
credit_receiver	Protocol	1/31	1.0	mutant_18	protocol_inv.
fifo_flops	Protocol	1/31	1.0	mutant_23	protocol_inv.
cdc_fifo_flops_push_credit	Protocol	0/31	0.0	None	protocol_inv.
AVERAGE / TOTAL	10 problems	9/10	0.9	--	--

Analysis

Why did the solved cases work? Combinational modules (enc_bin2gray, enc_bin2onehot, ecc_sed_encoder, shift_left, shift_right) map directly to mathematical formulas. The oracle simply needs to verify the output mapping, which Codex generates reliably. Sequential modules (lfsr, counter) have explicit cycle-by-cycle state transitions in the spec - the cycle_accurate oracle style generates precise assertions that match the spec exactly.

Protocol breakthrough in Phase 3: The credit_receiver and fifo_flops modules were both failing in the Phase 2 batch run (0.0 score). After adding the protocol-specific prompt with explicit instructions to avoid exact latency and queue-depth assumptions, and the guardrail check that rejects over-specified testbenches, both modules now produce Score = 1.0.

Why does cdc_fifo_flops_push_credit still fail?

This module has a massively complex specification describing a multi-domain CDC FIFO with credit-based flow control. The raw spec text is large enough to cause LLM context overflow and timeouts. Even when generation completes, the oracle invariably invents exact synchronization latency assumptions that the correct implementation does not satisfy. The testbench rejects all 31 mutants including the correct one. This is a semantic oracle failure driven by spec ambiguity, not a tooling or syntax problem.

9. Challenges, Limitations, and Improvements

Key Challenges

Protocol oracle generation: The hardest problem is not generating syntactically valid Verilog - the system does that reliably. The hard problem is generating a semantically correct oracle for modules with timing ambiguity. When the spec says 'credits are returned after consumption' without specifying exactly how many cycles later, Codex tends to invent a specific cycle count. That invented assumption is almost always wrong for the correct implementation.

Context-window limitations: The largest benchmark spec (cdc_fifo_flops_push_credit) exceeds practical context limits. Even with JSON compression in Stage 1, the aggregated context for generation is too large, leading to timeouts and degraded output quality.

Scalability: Each problem run requires multiple LLM calls (parse + plan + generate + possible repair), each with a 600-second timeout. Full 10-problem runs take 30-90 minutes. This limits iteration speed during development.

Limitations

Protocol-heavy modules remain a genuine research challenge. The system cannot leverage any information beyond what Codex can infer from the spec, which means semantic oracle failures for complex timing specifications are not easily addressable through engineering shortcuts alone.

Future Improvements

Property synthesis with simulation feedback: Rather than generating the full oracle in one shot, an iterative approach could generate candidate assertions, simulate each against all 31 mutants, observe which mutants pass/fail, and use that signal to prune over-constraining assertions. This would let the agent learn from simulation rather than relying entirely on prompt engineering to get the oracle right the first time.

Spec summarization for large context: For the largest specs, a pre-processing stage could summarize only the interface-level guarantees before passing context to generation, reducing token usage while preserving the most critical verification-relevant information.

10. AI Usage and Insights

AI (Codex / gpt-5.4) is the computational core of the system. It performs four distinct roles: spec parsing, test planning, testbench generation, and repair. Each role has different reliability characteristics.

Where AI works well:

Codex is reliable for extracting structured information from well-formed specs (Stage 1), generating combinational oracle formulas, and producing cycle-accurate state machine assertions for modules with explicit state transitions. In these cases the LLM output is correct on the first attempt with no repair needed.

Where AI fails:

The primary failure mode is overconfident oracle generation for protocol modules. Codex reads 'credit-based flow control' and generates an assertion like 'credit must return exactly 3 cycles after consumption.' That assumption is plausible but not stated in the spec. When it is wrong, the entire testbench fails. This is a hallucination of a specific constraint rather than a hallucination of code syntax.

Where human intervention was required:

The key human contributions were: (1) designing the oracle-planning fields in the parse prompt (guaranteed vs. ambiguous vs. forbidden), (2) designing the oracle-style routing logic, (3) writing the guardrail check that detects specific unsafe patterns in generated testbenches, and (4) writing the repair prompt to target oracle simplification rather than blind regeneration. None of these could be derived automatically - they required understanding of why the system was failing.

Decision-making role of AI:

At no point does the agent make autonomous decisions about design correctness. AI is used as a generator and transformer - it translates structured inputs into structured outputs. The oracle-style routing, guardrail checks, and repair triggers are all deterministic Python code, not LLM decisions. This is intentional: it keeps the system interpretable and debuggable.

11. Experience with AI

Using AI as a core component of a verification pipeline - rather than as a code-completion assistant - required a fundamentally different engineering mindset. The most important lesson was that prompt design is a form of specification writing. Vague prompts produce vague outputs. The system improved most when the prompts became more precise about what the output should NOT contain, not just what it should contain. The addition of forbidden_assumptions and unsafe_checks fields - which tell the model what to avoid - was more impactful than any positive instruction added.

Impact on productivity:

AI accelerated certain parts of development significantly. Boilerplate Verilog generation, repeated testbench structure, and handling of standard interface patterns (clock/reset/output checking) were all straightforward for Codex. The pipeline went from concept to working prototype much faster than would be possible with manual testbench writing.

Limitations encountered during development:

The model's confidence is uncalibrated. It presents invented timing constraints with the same confidence as directly-stated ones. There is no built-in signal for 'I am not sure about this assumption.' This means the engineering burden falls on the prompt designer to anticipate which assumptions the model is likely to hallucinate and explicitly forbid them.

What we learned about using AI as an engineer:

AI works best when given a precise, structured input format and asked to produce a precise, structured output. The more open-ended the task, the more the model will fill in details from training distribution rather than from the actual specification. The agent pipeline architecture - where the LLM is one component in a structured loop with EDA tool feedback - is a more robust pattern than treating the LLM as an end-to-end oracle. The simulation feedback loop is what makes the system self-correcting rather than dependent on a single lucky prompt.